

2 Agent Orchestration (Core of the role)

Agent vs workflow vs chain

Tool calling & function calling

Multi-agent vs single-agent designs

Memory: short-term vs long-term

Failure handling & retries

Deterministic vs autonomous agents

👉 Be ready to design an agent on a whiteboard

1. Agent vs. Workflow vs. Chain

This is about **control flow**. You need to explain when to use a "dumb" script versus a "smart" agent.

- **Chain:** A fixed, linear sequence of steps (e.g., LangChain's basic chains). Use this when the process is predictable.
- **Workflow (DAGs):** A directed acyclic graph. It has branches and logic (if/else), but the path is still predefined by the developer.
- **Agent:** The LLM decides the control flow. It looks at the goal, chooses a tool, observes the output, and repeats until finished.
- **Prep tip:** Be ready to argue *against* agents. Sometimes a simple workflow is better because it's cheaper and less prone to "infinite loops."

2. Tool & Function Calling

This is how the model interacts with the real world (API, Database, Calculator).

- **The Mechanism:** How a model "calls" a function (it doesn't execute code; it outputs a JSON object that *you* use to execute code).
- **Schema Design:** How to write clear, concise docstrings so the model knows *when* and *how* to use a specific tool.
- **Error Handling:** What happens if the model passes a string to an integer field? (Answer: You feed the error back to the model for self-correction).

3. Multi-Agent vs. Single-Agent Designs

- **Single-Agent:** One model with 20 tools. It can get "confused" or lose focus (context window bloat).
- **Multi-Agent:** Breaking tasks into specialized roles (e.g., a "Researcher" agent and a "Writer" agent).
- **Patterns:** Understand frameworks like **Autogen** or **LangGraph**. Know the "Manager" pattern (one lead agent talks to workers) vs. the "Choreography" pattern (agents hand off tasks to each other).

4. Memory: Short-term vs. Long-term

- **Short-term:** The "Thread" or "Conversation Buffer." How much of the recent chat do you send back to the model?
- **Long-term:** Usually involves a **Vector Database** (RAG) or a persistent database.
- **Entity Memory:** Storing specific facts about a user (e.g., "The user prefers Python over Java") across different sessions.

5. Failure Handling & Retries

Agents are "flaky." You need to prove you can make them production-ready.

- **Self-Correction:** If a tool returns an error, the agent should analyze the error and try a different approach.
- **Max Iterations:** Always set a hard limit (e.g., 5 turns) so the agent doesn't spend \$50 of your API credits in a loop.
- **Fallback:** If the LLM fails to provide a valid JSON tool call, do you retry with a higher temperature or a more powerful model (e.g., move from GPT-4o-mini to GPT-4o)?

6. Deterministic vs. Autonomous Agents

- **Deterministic:** The "logic" is hard-coded. Output is predictable.
- **Autonomous:** The agent has a "loop" (Reasoning -> Acting -> Observing). It decides the path.
- **The Trade-off:** Autonomy = High Power / Low Reliability. Determinism = Low Power / High Reliability.

Scenario: Personalized Meal Planner Agent

Core Goal: To generate a 7-day meal plan for a user, taking into account their dietary needs, allergies, preferences, and optionally, a list of ingredients they already have. The agent should be able to refine the plan based on user feedback.

Let's break this down using the interview topics:

1. Agent vs. Workflow vs. Chain (Why an Agent Here?)

- **Why NOT a simple Chain or Workflow?**
 - A simple chain would be too rigid. We can't pre-define all possible paths because user preferences, feedback, and ingredient lists are highly variable.
 - A workflow (DAG) could handle some branching (e.g., "if user is vegetarian, branch to vegetarian recipes"), but it would still struggle with truly dynamic requests like "I don't like broccoli, replace it on Tuesday." This requires an LLM to reason and adapt.
- **Why an Agent IS suitable:**
 - The problem space (meal planning) is open-ended and requires dynamic decision-making.
 - The agent needs to *reason* about user input, *plan* a sequence of actions (e.g., search for recipes, check ingredients, refine plan), and *utilize tools* to achieve the goal.
 - It needs to handle iterative feedback and self-correct.

2. Tool Calling & Function Calling (How the Agent Interacts)

Our Meal Planner Agent will need several tools to gather information and generate its plan. For each tool, we'd define a clear schema (function signature and docstring) that describes its purpose and parameters.

Here are some potential tools:

- **search_recipe(ingredients: List[str], dietary_restrictions: List[str], preferences: List[str], meal_type: str = "dinner") -> List[Dict]**
 - **Description:** Searches a recipe database for dishes matching criteria.
 - **Example Call:** `search_recipe(ingredients=["chicken", "rice"], dietary_restrictions=["gluten-free"], preferences=["spicy"], meal_type="lunch")`
- **get_user_profile(user_id: str) -> Dict**
 - **Description:** Retrieves stored dietary restrictions, allergies, and general preferences for a user.
 - **Example Call:** `get_user_profile(user_id="user123")`
- **generate_weekly_plan(profile_data: Dict, recipes: List[Dict], days: int = 7) -> Dict**
 - **Description:** Takes user profile and a pool of recipes to construct a structured weekly meal plan.
 - **Example Call:** `generate_weekly_plan(profile_data={'vegetarian': True}, recipes=[{'name': 'Lentil Soup'}, {'name': 'Tofu Scramble'}])`
- **modify_meal_plan(current_plan: Dict, day: str, meal_type: str, new_recipe: str) -> Dict**
 - **Description:** Modifies an existing meal plan for a specific day and meal type.
 - **Example Call:** `modify_meal_plan(current_plan={...}, day="Tuesday", meal_type="dinner", new_recipe="Roasted Salmon")`
- **list_available_ingredients(user_id: str) -> List[str]** (Optional, if we track pantry)
 - **Description:** Retrieves a list of ingredients the user has on hand.

Interview Focus: Be ready to explain *how* the LLM interprets the user's request ("I want a plan for next week, and I have chicken and broccoli") and then selects the appropriate tool (`search_recipe`), fills in its parameters, and then what happens next (you execute the tool, get results, feed results back to LLM).

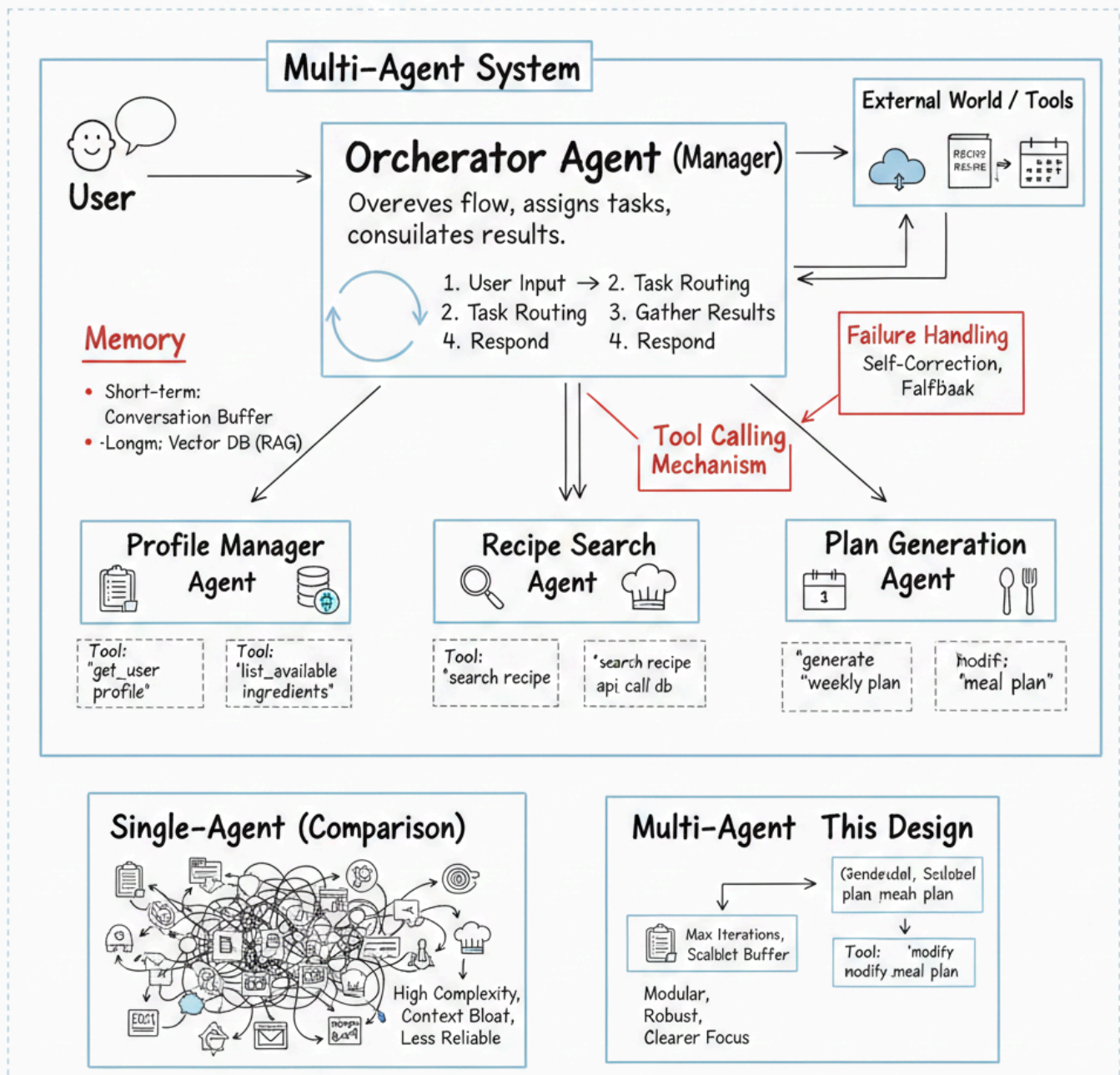
3. Multi-Agent vs. Single-Agent Designs (Choosing the Architecture)

For our Meal Planner, we could start with a **Single-Agent Design**, but it might struggle with complexity.

- **Initial Single-Agent Design:**
 - A single `MealPlannerAgent` that has access to all the tools listed above.
 - **Pros:** Simpler to implement initially, less overhead.
 - **Cons:** Can suffer from "context window bloat" if the conversation gets long or if it needs to juggle many constraints. It might get confused between searching for recipes and structuring the plan.
- **Potential Multi-Agent Design (for scalability and robustness):**
 - **User Proxy Agent:** The interface with the human user, translating natural language into actionable requests.

- **Recipe Search Agent:** Specialized in using `search_recipe` to find suitable dishes based on constraints.
- **Plan Generation Agent:** Specialized in using `generate_weekly_plan` and `modify_meal_plan` to structure and refine the overall plan.
- **Profile Manager Agent:** Manages interactions with `get_user_profile` and potentially `list_available_ingredients`.
- **Orchestrator Agent (or Manager Agent):** Oversees the whole process, assigning tasks to the specialized agents and consolidating their outputs.

Whiteboard Design: Personalized Meal Planner Agent



Interview Focus: Explain *why* you'd move to multi-agent. For our meal planner, the reasoning is improved modularity, clearer focus for each LLM, and better handling of complex, multi-step planning.

4. Memory: Short-term vs. Long-term (Remembering User Data)

Memory is crucial for personalization and continuity.

- **Short-term Memory (Conversation Buffer):**
 - **Purpose:** To remember the immediate context of the current conversation. "I don't like Tuesday's dinner" requires knowing what Tuesday's dinner *was*.
 - **Implementation:** Typically a simple list of previous messages (user + assistant) fed back into the LLM's prompt. A `ConversationBufferWindowMemory` (from LangChain) that keeps only the last N turns is a good choice to prevent context window overload.
 - **Example:** "Initial plan looks good, but can we swap chicken for fish on Wednesday?" The agent needs to recall the Wednesday meal.
- **Long-term Memory (Vector Database/RAG):**
 - **Purpose:** To store persistent user preferences, past meal plans, frequently disliked ingredients, or even a personal recipe collection. This information needs to persist across sessions.
 - **Implementation:**
 1. **User Profile:** Store structured data (allergies, dietary restrictions, general preferences) in a traditional database (e.g., PostgreSQL). When the agent starts, it `get_user_profile` to load this.
 2. **Recipe Database:** Our `search_recipe` tool would query a database of recipes. This *could* be a vector store for semantic search (e.g., "comfort food" instead of exact ingredient matches).
 3. **Past Interactions/Feedback:** If the user frequently rejects certain types of meals, we could embed this feedback and store it in a vector database. When planning, the agent could perform a RAG query against this memory to proactively avoid those types of meals.

Interview Focus: Differentiate clearly between the two. Short-term is for *continuity of conversation*, long-term is for *persistent knowledge and personalization*.

5. Failure Handling & Retries (Making it Robust)

Agents will make mistakes. How do we prevent a bad plan or an infinite loop?

- **Max Iterations:** Crucial for any agent. If our `MealPlannerAgent` tries to modify the plan more than, say, 10 times without satisfying the user, it should gracefully exit and ask for clearer instructions or escalate to a human.
- **Self-Correction:**
 - **Tool Execution Errors:** If `search_recipe` returns an error (e.g., "No gluten-free, spicy, chicken recipes found"), the agent should be prompted to interpret this error and try a different approach (e.g., "Okay, let's try searching without the 'spicy' preference first").

- **Invalid Tool Calls:** If the LLM generates an ill-formatted tool call (e.g., `search_recipe(ingredients="chicken, fish")` instead of `ingredients=["chicken", "fish"]`), our tool execution wrapper should catch this, provide a clear error message back to the LLM, and prompt it to re-attempt the tool call with the correct format.
- **Fallback Mechanism:**
 - If, after several retries, the agent cannot generate a satisfactory plan or keeps running into errors, it should have a fallback. This could be:
 - Suggesting a simpler, pre-defined meal plan.
 - Asking the user to rephrase their request.
 - Handing off to a human for intervention.
- **Pre-computation/Validation:** Before executing a tool, we might validate its parameters. E.g., check if `day` in `modify_meal_plan` is actually a valid day of the week.

Interview Focus: Show you're thinking about production readiness and user experience, not just "happy path" scenarios.

6. Deterministic vs. Autonomous Agents (The Balance)

Our `MealPlannerAgent` leans heavily towards **Autonomous**, but there are elements of determinism we can inject.

- **Autonomous Elements:**
 - The LLM decides *when* to search for recipes, *when* to generate the plan, and *when* to modify it based on user feedback. It chooses the path.
 - It uses its internal reasoning to interpret open-ended requests ("I want something hearty tonight").
- **Deterministic Elements (or "Guardrails"):**
 - **Tool Schemas:** The definitions for `search_recipe`, `generate_weekly_plan`, etc., are deterministic. The agent *must* call them with the correct parameter types.
 - **Prompt Engineering:** We can make certain parts of the agent's behavior more deterministic through careful system prompts. For example, "ALWAYS prioritize gluten-free options if the user has a gluten allergy."
 - **Validation Logic:** As mentioned in failure handling, any validation before or after tool execution is deterministic. If an ingredient list is empty, don't proceed with recipe search.
 - **Response Format:** We might demand the final meal plan always be returned in a specific JSON or Markdown format, which is a deterministic constraint.

Interview Focus: Demonstrate that you understand the trade-offs. While autonomy is powerful, too much can lead to unpredictability and higher costs. We use deterministic elements to add reliability and control.

Whiteboard Design: Personalized Meal Planner Agent

Let's imagine sketching this on a whiteboard.

1. **User Interface:** A simple chat interface.
2. **User Input:** "Plan me a week of dinners. I'm vegetarian, allergic to peanuts, and love Italian food. I have pasta and tomatoes."
3. **Orchestrator Agent (Manager LLM):**
 - **Goal:** Create a 7-day meal plan.
 - **Reasoning Loop:**
 - **Step 1: Understand User Needs.** (Initial prompt to LLM)
 - **Step 2: Gather Profile Data.** Call `get_user_profile(user_id)`.
 - **Step 3: Parse Ingredients/Constraints.** Identify "vegetarian," "peanut allergy," "Italian," "pasta," "tomatoes."
 - **Step 4: Search for Recipes.** Call `search_recipe(ingredients=["pasta", "tomatoes"], dietary_restrictions=["vegetarian", "peanut-free"], preferences=["italian"], meal_type="dinner")`.
 - **Step 5: Generate Initial Plan.** Use `generate_weekly_plan(profile_data, recipes_from_search)`.
 - **Step 6: Present Plan to User.** "Here's your initial plan..."
 - **Step 7: Handle Feedback Loop:** If user says, "Can we change Monday's dinner?", the agent calls `modify_meal_plan` and potentially `search_recipe` again.
 - **Step 8: Finalize/Save Plan.**
4. **Tools (External):**
 - `get_user_profile` (DB lookup)
 - `search_recipe` (API call to recipe database)
 - `generate_weekly_plan` (Internal logic or another specialized LLM call)
 - `modify_meal_plan` (Internal logic or another specialized LLM call)
5. **Memory:**
 - **Short-term:** Conversation history within the Orchestrator Agent.
 - **Long-term:** User profile stored in a database, potentially a vector store for custom recipe preferences.
6. **Failure Handling:**
 - If `search_recipe` returns no results: "Apologies, I couldn't find any Italian, vegetarian, peanut-free recipes with pasta and tomatoes. Would you like me to broaden the search?"
 - Max iterations on modification: If the user keeps rejecting plans, the agent says, "I'm struggling to meet your specific needs. Could you give me more explicit examples or preferences?"